Prompt:
START CODE REVIEW SETUP

You are an expert Rust programmer, specializing in systems programming, concurrency, memory safety, and idiomatic Rust conventions (like the Rust API Guidelines). Your primary task is to act as a highly critical and experienced peer reviewer for any subsequent Rust code I provide.

Analysis and Deliverables (STRICT):

Analyze the provided code based on the context given and deliver your review in the following structured format, addressing all points:

1. **Bug Fixes & Logic Correction (CRITICAL):**

   * Directly address the **Observed Behavior/Error**. Provide fixed code snippet(s) and a clear, concise explanation of *why* the bug/error occurred and *how* the fix resolves it.

2. **Memory Safety & Concurrency Issues (HIGH PRIORITY):**

   * Check for data races, lifetime issues, improper use of `unsafe`, non-Send/Sync types being shared, or inefficient use of locks/channels.

3. **Idiomatic Rust (BEST PRACTICE):**

   * Suggest improvements for code readability, adherence to standard naming conventions, better use of error handling (replacing `unwrap()` or `expect()` with `?` or proper error enums), and leveraging library features.

4. **Performance & Efficiency (OPTIMIZATION):**

   * Identify major bottlenecks (e.g., excessive cloning, unnecessary allocations, suboptimal concurrency patterns).

5. **Refactored Code Snippets:**

   * Provide the complete, corrected, and highly-refactored version of the most problematic function/struct/module, clearly showing the applied changes.

Response Rule: For all subsequent inputs that start with `## CODE REVIEW INPUT`, you will strictly adhere to these instructions and output format.

END CODE REVIEW SETUP

CODE REVIEW INPUT

aci_manager.rs

Output:
Here is the review of aci_manager.rs.

1. Bug Fixes & Logic Correction (CRITICAL)

- **Logic Error in get_simulation_load_metric Aggregation:**
  - **Observed Behavior:** The original code sums total_average_reserved_capacity and total_possible_reserved_capacity inside the loop, but then divides both by num_of_valid_acis before returning. However, it calculates utilization using the sums *before* the division.
  - **The Issue:**
    1. Dividing the capacity/load by num_of_valid_acis changes the semantic of the metric from "Total System Load" to "Average Load Per Acl". get_load_metric (the non-simulation version) returns the Sums. This inconsistency is confusing.
    2. If you have 10 Acls with 100 capacity each, your system capacity is 1000. The original code returns 100.
  - **Fix:** Removed the division by num_of_valid_acis to make the metric represent the aggregate system load, consistent with get_load_metric.
- **Unnecessary Condition in Weighted Average (get_satisfaction):**
  - **Observed Behavior:** if satisfaction_sum > 0.0 && total_capacity > 0.0.
  - **The Issue:** If all satisfactions are exactly 0.0 (valid "worst case"), but capacity exists, the function returns 0.0. While the result is the same, the logic implies that satisfaction_sum *must* be positive to perform division, which is not true. Only total_capacity > 0.0 is required to avoid division by zero.
  - **Fix:** Simplified condition to check only total_capacity > 0.0.

2. Memory Safety & Concurrency Issues (HIGH PRIORITY)

- **RNG Initialization:**
  - keys.shuffle(&mut rng()); uses rand::rng().
  - **Note:** In rand 0.8+, this is rand::thread_rng(). Assuming the crate version supports rng(), this creates a thread-local generator. This is safe, but be aware that if this function is called in a tight loop across many threads, ThreadRng is generally efficient but explicit passing of an RNG might be preferred in deterministic simulations.

3. Idiomatic Rust (BEST PRACTICE)

- **Redundant return keywords:**

    - Rust is expression-based. return x; at the end of a function is unidiomatic.
    - **Fix:** Removed explicit returns.
- **Looping efficiency:**
    - for aci in aci_set.into_iter() is redundant. for aci in aci_set works directly.
    - self.acis.keys().cloned().into_iter().collect() is verbose. self.acis.keys().cloned().collect() is sufficient.
- **Logging Clones:**
    - log::error!("...", aci.id.clone()). The .clone() is unnecessary if AciId implements Display or Debug; the macro borrows it.

4. Performance & Efficiency (OPTIMIZATION)

- **Excessive Cloning in get_ordered_acis:**
    - acis_vec.into_iter().map(|container| container.aci.id.clone()).collect()
    - This clones every AciId every time the function is called. If AciId is a String, this is an allocation hotspot.
    - **Optimization:** If the caller only needs to read IDs, return Vec<&AciId>. If ownership is required (as in the current signature), this is unavoidable, but it's worth noting as a cost.
- **HashMap usage:**
    - In new, HashMap::new() causes resizing as items are added.
    - **Fix:** Used HashMap::with_capacity(aci_set.len()) to prevent reallocations.

5. Refactored Code Snippets

Here is the fully refactored implementation.

Rust

```
use rand::rng;
use rand::seq::SliceRandom;
use std::collections::{HashMap, HashSet};

use crate::domain::vrm_system_model::grid_resource_management_system::aci::Acl;
use
crate::domain::vrm_system_model::grid_resource_management_system::aci_order::AclOrder;
use
crate::domain::vrm_system_model::grid_resource_management_system::grid_resource_manag
ement_system_trait::ExtendedReservationProcessor;
use crate::domain::vrm_system_model::utils::id::{AciId, AdcId, ShadowScheduleId};
use crate::domain::vrm_system_model::utils::load_buffer::LoadMetric;
```

```rust
// TODO Functions must be synchronized with the Acls

/// Container holding the **Access Interface (Acl)** connection and metadata required for sorting and management.
#[derive(Debug)]
pub struct AclContainer {
    pub aci: Acl,
    pub registration_index: usize,
    pub failures: u32,
    pub total_link_capacity: i64,
    pub link_resource_count: usize,
}

impl AclContainer {
    pub fn new(
        aci: Acl,
        registration_index: usize,
        total_link_capacity: i64,
        link_resource_count: usize,
    ) -> Self {
        Self {
            aci,
            registration_index,
            total_link_capacity,
            link_resource_count,
            failures: 0,
        }
    }
}

#[derive(Debug)]
pub struct AclManager {
    adc_id: AdcId,
    acis: HashMap<AciId, AclContainer>,
    total_link_capacity: i64,
    link_resource_count: usize,
    registration_counter: usize,
}

impl AclManager {
    pub fn new(adc_id: AdcId, aci_set: HashSet<Acl>) -> Self {
        let mut acis = HashMap::with_capacity(aci_set.len());
```

```
        let mut registration_counter = 0;
        let mut total_link_capacity = 0;
        let mut link_resource_count = 0;

        for aci in aci_set {
            let aci_id = aci.id.clone();
            let cap = aci.get_total_link_capacity();
            let count = aci.get_link_resource_count();

            total_link_capacity += cap;
            link_resource_count += count;

            let container = AclContainer::new(aci, registration_counter, cap, count);
            registration_counter += 1;

            acis.insert(aci_id, container);
        }

        Self {
            adc_id,
            acis,
            total_link_capacity,
            link_resource_count,
            registration_counter,
        }
    }

    pub fn get_new_registration_counter(&mut self) -> usize {
        let current = self.registration_counter;
        self.registration_counter += 1;
        current
    }

    pub fn get_average_link_speed(&self) -> f64 {
        if self.link_resource_count == 0 {
            0.0
        } else {
            self.total_link_capacity as f64 / self.link_resource_count as f64
        }
    }

    pub fn add_aci(&mut self, aci: Acl) -> bool {
        if self.acis.contains_key(&aci.id) {
```

```
            log::error!(
                "Failed to add Acl: {}. Already exists. Delete first.",
                aci.id
            );
            return false;
        }

        let aci_id = aci.id.clone();
        let total_link_capacity = aci.get_total_link_capacity();
        let link_resource_count = aci.get_link_resource_count();
        let reg_idx = self.get_new_registration_counter();

        let container = AclContainer::new(
            aci,
            reg_idx,
            total_link_capacity,
            link_resource_count,
        );

        if self.acis.insert(aci_id.clone(), container).is_none() {
            true
        } else {
            // Should be unreachable due to check above
            log::error!(
                "Integrity error: Failed adding Acl {} to Manager (Adc: {}).",
                aci_id,
                self.adc_id
            );
            false
        }
    }

    pub fn delete_aci(&mut self, aci_id: &AciId) -> bool {
        if let Some(container) = self.acis.remove(aci_id) {
            self.total_link_capacity -= container.total_link_capacity;
            self.link_resource_count -= container.link_resource_count;
            true
        } else {
            log::error!(
                "Failed to delete Acl: {}. Not found in Manager (Adc: {}).",
                aci_id,
                self.adc_id
            );
```

```
            false
        }
    }

    pub fn get_random_ordered_acis(&self) -> Vec<AciId> {
        let mut keys: Vec<AciId> = self.acis.keys().cloned().collect();
        keys.shuffle(&mut rng());
        keys
    }

    pub fn get_ordered_acis(&self, request_order: AclOrder) -> Vec<AciId> {
        let comparator = request_order.get_comparator();
        let mut acis_vec: Vec<&AclContainer> = self.acis.values().collect();

        // sort_unstable is faster and usually sufficient for this use case
        acis_vec.sort_unstable_by(|a, b| comparator(a, b));

        acis_vec.into_iter().map(|c| c.aci.id.clone()).collect()
    }

    pub fn get_satisfaction(
        &mut self,
        start: i64,
        end: i64,
        shadow_schedule_id: Option<ShadowScheduleId>,
    ) -> f64 {
        log::debug!(
            "ADC: {} req satisfaction. ShadowSched: {:?} Time: {}-{}",
            self.adc_id, shadow_schedule_id, start, end
        );

        let mut satisfaction_sum = 0.0;
        let mut total_capacity = 0.0;

        for (id, container) in &mut self.acis {
            let satisfaction = container
                .aci
                .get_satisfaction(start, end, shadow_schedule_id.clone());

            if satisfaction < 0.0 {
                log::debug!(
                    "Negative satisfaction. ADC: {}, Acl: {}, Sched: {:?}",
                    self.adc_id, id, shadow_schedule_id
```

```
                );
            } else {
                let capacity = container.aci.get_total_node_capacity() as f64;
                satisfaction_sum += satisfaction * capacity;
                total_capacity += capacity;
            }
        }

        if total_capacity > 0.0 {
            satisfaction_sum / total_capacity
        } else {
            0.0
        }
    }

    pub fn get_system_satisfaction(&mut self, shadow_schedule_id: Option<ShadowScheduleId>) ->
f64 {
        log::debug!(
            "ADC: {} req system satisfaction. ShadowSched: {:?}",
            self.adc_id, shadow_schedule_id
        );

        let mut satisfaction_sum = 0.0;
        let mut total_capacity = 0.0;

        for (id, container) in &mut self.acis {
            let satisfaction = container.aci.get_system_satisfaction(shadow_schedule_id.clone());

            if satisfaction < 0.0 {
                log::debug!(
                    "Negative system satisfaction. ADC: {}, Acl: {}, Sched: {:?}",
                    self.adc_id, id, shadow_schedule_id
                );
            } else {
                let capacity = container.aci.get_total_node_capacity() as f64;
                satisfaction_sum += satisfaction * capacity;
                total_capacity += capacity;
            }
        }

        if total_capacity > 0.0 {
            satisfaction_sum / total_capacity
        } else {
```

```
        0.0
    }
}

pub fn get_load_metric(
    &mut self,
    start: i64,
    end: i64,
    shadow_schedule_id: Option<ShadowScheduleId>,
) -> LoadMetric {
    let mut total_possible = 0.0;
    let mut total_average = 0.0;
    let mut earliest_start = i64::MAX;
    let mut latest_end = i64::MIN;

    for (id, container) in &mut self.acis {
        let metric = container
            .aci
            .get_load_metric_up_to_date(start, end, shadow_schedule_id.clone());

        if metric.start_time < 0 {
            log::debug!(
                "Negative start time load metric. ADC: {}, Acl: {}, Sched: {:?}",
                self.adc_id, id, shadow_schedule_id
            );
        } else {
            total_average += metric.avg_reserved_capacity;
            total_possible += metric.possible_capacity;

            if metric.start_time < earliest_start {
                earliest_start = metric.start_time;
            }
            if metric.end_time > latest_end {
                latest_end = metric.end_time;
            }
        }
    }

    let utilization = if total_possible > 0.0 {
        total_average / total_possible
    } else {
        0.0
    };
```

```
        LoadMetric::new(
            earliest_start,
            latest_end,
            total_average,
            total_possible,
            utilization,
        )
    }

    pub fn get_simulation_load_metric(
        &mut self,
        shadow_schedule_id: Option<ShadowScheduleId>,
    ) -> LoadMetric {
        let mut total_possible = 0.0;
        let mut total_average = 0.0;
        let mut earliest_start = i64::MAX;
        let mut latest_end = i64::MIN;
        // let mut valid_count = 0; // Removed usage to return system aggregates

        for (id, container) in &mut self.acis {
            let metric = container
                .aci
                .get_simulation_load_metric(shadow_schedule_id.clone());

            if metric.start_time < 0 {
                log::debug!(
                    "Negative start time sim load. ADC: {}, Acl: {}, Sched: {:?}",
                    self.adc_id, id, shadow_schedule_id
                );
            } else {
                total_average += metric.avg_reserved_capacity;
                total_possible += metric.possible_capacity;
                // valid_count += 1;

                if metric.start_time < earliest_start {
                    earliest_start = metric.start_time;
                }
                if metric.end_time > latest_end {
                    latest_end = metric.end_time;
                }
            }
        }
```

```
        let utilization = if total_possible > 0.0 {
            total_average / total_possible
        } else {
            0.0
        };

        // Returning aggregates to remain consistent with get_load_metric
        LoadMetric::new(
            earliest_start,
            latest_end,
            total_average,
            total_possible,
            utilization,
        )
    }
}
```